<!-- به نام خدا -->

مهندسی اینترنت، فصل ۱: پروتکل‌ها

یک تراکنش HTTP

جلسهٔ ۲: ساختار درخواست و پاسخ، و مرجعی که معنای آن‌ها را تعیین می‌کند

parse · HSTS · DNS · اتصال · TLS ◌ · HTTP ●

محمد بصیرزاده

دانشگاه شهید چمران اهواز، دانشکدهٔ مهندسی، پاییز ۱۴۰۵

اگر خودتان پروتکل را طراحی کنید، درخواست چه اجزایی لازم دارد؟

اتصال به سرور برقرار است. کلاینت باید در پیام درخواست بگوید چه می‌خواهد.

الف) فقط نام فایل

ب) نام فایل و نوع عمل

ج) عمل، منبع، نسخه و اطلاعات تکمیلی

د) فقط نشانی IP

اگر خودتان پروتکل را طراحی کنید، درخواست چه اجزایی لازم دارد؟

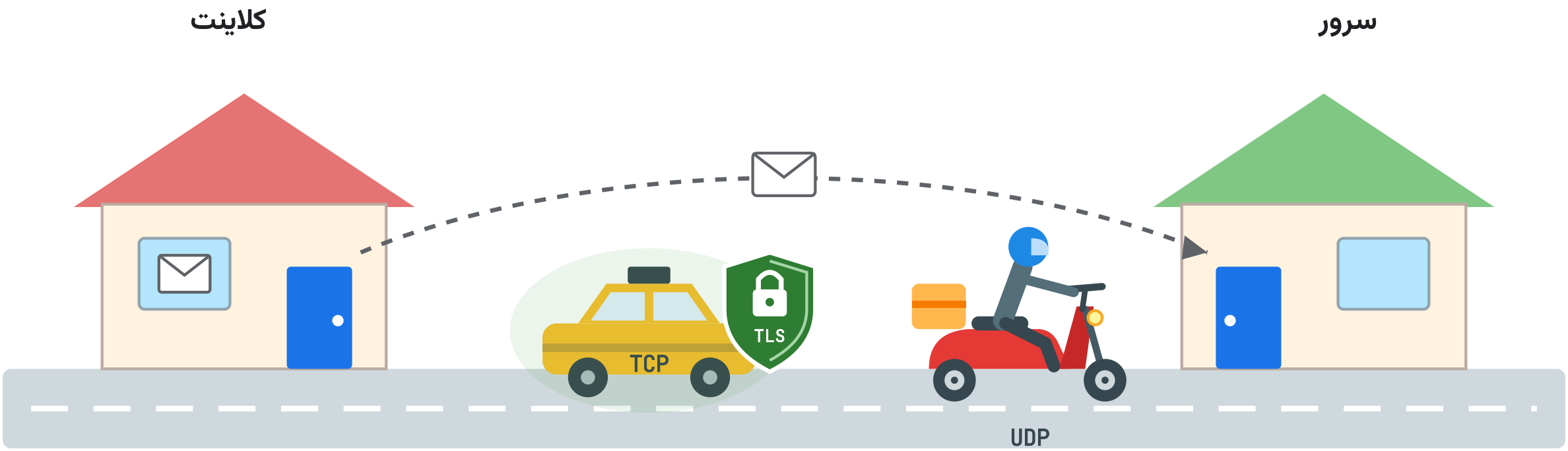
اتصال به سرور برقرار است. کلاینت باید در پیام درخواست بگوید چه می‌خواهد.

الف) فقط نام فایل	**ب)** نام فایل و نوع عمل	**ج)** عمل، منبع، نسخه و اطلاعات تکمیلی	**د)** فقط نشانی IP

پاسخ: ج) HTTP همین را دارد: متد و هدف و نسخه در خط درخواست، و اطلاعات تکمیلی در فیلدهای header.
نسخهٔ نخست HTTP فقط نام فایل را می‌فرستاد.

در این جلسه طراحی واقعی HTTP را با همین فهرست مقایسه می‌کنیم.

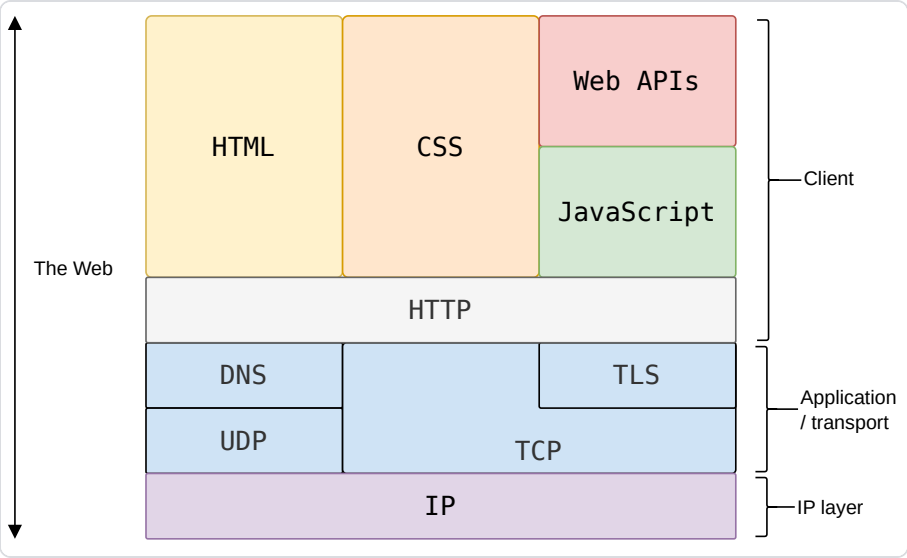
پیام از یک برنامه به برنامهٔ دیگر چگونه می‌رسد؟



> **لایهٔ کاربرد نامه را می‌نویسد و می‌فهمد؛ رساندن آن بر عهدهٔ لایهٔ انتقال است.**

HTTP در کدام لایه قرار دارد و چه نقشی دارد؟

- پروتکل لایهٔ کاربرد وب است و **ساختار پیام‌ها و ترتیب تبادل آن‌ها** را میان کلاینت و سرور تعریف می‌کند؛ HTML و مرورگر و سرور اجزای دیگر وب‌اند
- انتقال مطمئن را از TCP می‌گیرد و نگران گم شدن یا جابه‌جایی داده نیست؛ HTTP/3 روی QUIC (جلسهٔ ۴)
- stateless است: اگر کلاینت یک object را دو بار بخواهد، سرور آن را دوباره می‌فرستد (جلسهٔ ۳)



جایگاه HTTP در پشتهٔ پروتکل‌ها

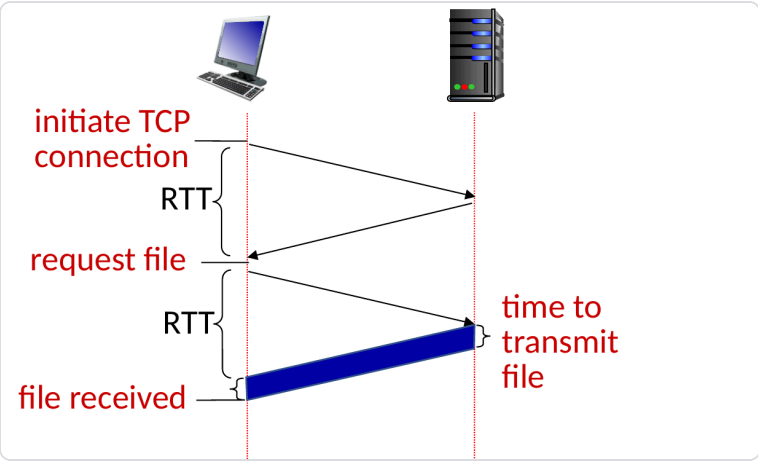
منبع: MDN, Mozilla Contributors · CC BY-SA 2.5

HTTP ساختار و ترتیب پیام‌ها را تعریف می‌کند و هر درخواست را مستقل از درخواست‌های قبلی می‌بیند.

یک پروتکل لایهٔ کاربرد چه چیزهایی را تعریف می‌کند؟

چهار محور هر پروتکل (Kurose و Ross، بخش ۲.۱.۵):

محور	در HTTP
نوع پیام	درخواست و پاسخ
نحو	خط آغاز، header، خط خالی، بدنه
معنا	**متد و کد وضعیت** و فیلدها
قواعد	تکرار، redirect، مذاکره



یک تراکنش در زمان؛ هر object صفحه یک تراکنش دارد

منبع: Kurose & Ross, Figure 2.7 · author slides

پروتکل یعنی توافق بر نوع و نحو و معنا و قواعد پیام‌ها.

بخش ۱ از ۶

پیام: روی اتصال چه فرستاده می‌شود؟

تا اینجا: جایگاه HTTP و اجزای یک تراکنش را دیدیم.

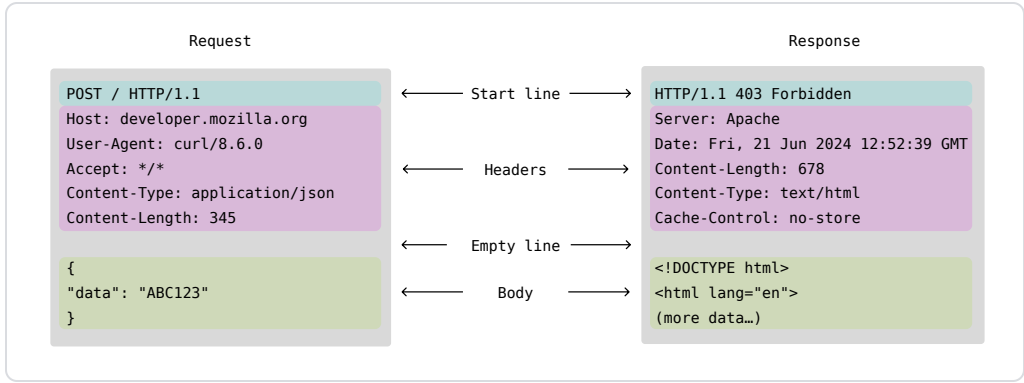
سؤال این بخش

روی اتصال دقیقاً چه بایت‌هایی منتقل می‌شود و گیرنده پایان پیام را از کجا تشخیص می‌دهد؟

02-http-transaction.todo

- [x] پیام
- [ ] header و مذاکره
- [ ] متد
- [ ] کد وضعیت
- [ ] چالش‌ها
- [ ] جمع‌بندی

درخواست و پاسخ چه ساختار مشترکی دارند؟



ساختار مشترک پیام درخواست و پاسخ

منبع: MDN, Mozilla Contributors · CC BY-SA 2.5

```
request line : method SP target SP version CRLF
status line  : version SP status-code SP reason CRLF
header line  : field-name ":" SP value CRLF
blank line   : CRLF                (then the body, if any)
```

دو پیام یک قالب دارند؛ تفاوت آن‌ها در خط آغاز است.

چرا درخواست HTTP نام سایت را دوباره می‌فرستد؟

در تمرین ۱ دیدید که curl پس از برقراری اتصال، نام سایت را در فیلد `Host` نیز فرستاد. سرور نشانی IP و پورت را از خود اتصال می‌داند؛ این فیلد چه ضرورتی دارد؟

الف) امنیت ارتباط

ب) چند سایت روی یک IP

ج) cache

د) TLS

چرا درخواست HTTP نام سایت را دوباره می‌فرستد؟

در تمرین ۱ دیدید که curl پس از برقراری اتصال، نام سایت را در فیلد `Host` نیز فرستاد. سرور نشانی IP و پورت را از خود اتصال می‌داند؛ این فیلد چه ضرورتی دارد؟

الف) امنیت ارتباط	**ب)** چند سایت روی یک IP	**ج)** cache	**د)** TLS

پاسخ: ب) روی یک نشانی IP ممکن است هزاران سایت میزبانی شود (virtual hosting) و سرور تنها از فیلد `Host` درمی‌یابد کدام سایت درخواست شده است. طبق RFC 9112 §3.2 این فیلد در HTTP/1.1 الزامی است.

> **نشانی IP و پورت، میزبان را مشخص می‌کنند؛ نام سایت را تنها HTTP می‌شناسد.**

نرم‌افزار کلاینت کدام بخش پاسخ را می‌خواند؟

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 29
(خط خالی)
{"id":7,"items":["کتاب"]}
```

خط وضعیت
نوع بازنمایی
طول بدنه به بایت
پایان فیلدها
بدنه (body)

- Content-Length طول را به بایت می‌سنجد، نه به نویسه: «کتاب» چهار نویسه و **هشت بایت** در UTF-8 است
- نوع object را `Content-Type` تعیین می‌کند، نه پسوند فایل

> **نرم‌افزار عدد را می‌خواند، نه عبارت کنار آن را.**

پیام کجا تمام می‌شود؟

خط خالی پایان headerهاست؛ پایان بدنه را چه چیزی مشخص می‌کند؟

روش	پایان پیام	کجا
`Content-Length: 29`	بعد از دقیقاً ۲۹ بایت	درخواست و پاسخ
`Transfer-Encoding: chunked`	هر تکه طول خودش را دارد؛ تکهٔ صفر یعنی پایان	وقتی طول از اول معلوم نیست
بسته شدن اتصال	هرچه تا بسته شدن رسید	فقط پاسخ

اگر دو طرف این مرز را متفاوت تشخیص دهند چه می‌شود؟ بخش چالش‌ها.

> **در HTTP/1.1 مرز پیام را خود پیام اعلام می‌کند؛ خوانش متفاوت یعنی دو پیام متفاوت.**

همان درخواست در HTTP/2 چه شکلی دارد؟

خروجی `curl -v` برای یک درخواست، یک بار با HTTP/1.1 و یک بار با HTTP/2:

```
HTTP/1.1                                  HTTP/2
> GET /hello?x=1 HTTP/1.1                 * [HTTP/2] [1] [:method: GET]
> Host: 127.0.0.1:8767                    * [HTTP/2] [1] [:authority: 127.0.0.1:8766]
< HTTP/1.1 200 OK                         < HTTP/2 200
< Connection: keep-alive                  (no Connection, no Keep-Alive)
server saw: host=127.0.0.1:8767           server saw: host=-  :authority=127.0.0.1:8766
```

- در HTTP/2 خط درخواست به pseudo-headerهایی مانند `:method` و `:authority` تبدیل می‌شود؛ پاسخ reason phrase ندارد و فیلد `Connection` **مجاز نیست**

> **معنای درخواست تغییر نکرده است؛ تنها قالب ارسال آن تغییر کرده است.**

دموی زنده: HTTP/1.1 و HTTP/2

سؤال اگر همان درخواست را با HTTP/2 بفرستیم، سرور چه می‌بیند؟

مشاهده خروجی `curl -v` از سروری محلی که HTTP/1.1 و HTTP/2 بی‌رمز را پشتیبانی می‌کند

تصمیم کد سرور به reason phrase و فیلدهای مخصوص اتصال تکیه نکند

```
curl -sv http://127.0.0.1:8767/hello?x=1
curl -sv --http2-prior-knowledge http://127.0.0.1:8766/hello?x=1
```

خط‌هایی را که با `<` شروع می‌شوند با اسلاید قبل مقایسه کنید.

درخواست را خودتان بنویسید

به پورت ۸۰ یک سرور وصل شوید و درخواست را دستی تایپ کنید؛ پس از خط `Host` دو بار Enter بزنید (همان نمونهٔ Kurose و Ross، بخش ۲.۲.۳):

```
nc -C gaia.cs.umass.edu 80
GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu
```

- پاسخ همان قالب بخش ۱ را دارد: خط وضعیت، headerها، خط خالی، بدنه؛ با `HEAD` به‌جای `GET` فقط headerها برمی‌گردد
- گزینهٔ `-C` در Linux کلید Enter را به CRLF تبدیل می‌کند؛ در macOS گزینهٔ `-c` همین کار را می‌کند
- تمرین ۲ همین گفت‌وگو را از سمت سرور می‌سازد

HTTP سه نسخه دارد؛ چه چیزی در هر سه یکی است؟

لایه	سند	آنچه تعریف می‌کند
معنا	RFC 9110	متد، کد وضعیت، فیلدها، و قاعدهٔ رفتار با هرکدام
HTTP/1.1	RFC 9112	پیام متنی روی TCP
HTTP/2	RFC 9113	frame باینری و چند جریان روی یک اتصال TCP
HTTP/3	RFC 9114	frame باینری روی QUIC

تنها HTTP/1.x متنی است. تفاوت نسخه‌ها و علت وجود آن‌ها در جلسهٔ ۴ بررسی می‌شود؛ متد و کد وضعیت و header در هر سه نسخه معنای یکسان دارند.

> **معنای پیام در هر سه نسخه یکی است؛ ولی هر قالب ملاحظات خاص خود را دارد.**

بخش ۲ از ۶

header و مذاکره: بخش مشترک دو پیام

تا اینجا: قالب پیام را دیدیم؛ هر دو پیام فیلدهای header دارند.

سؤال این بخش

headerها چه می‌گویند، و کلاینت و سرور چگونه با آن‌ها بر سر شکل پاسخ توافق می‌کنند؟

02-http-transaction.todo

[x] پیام
[>] header و مذاکره ▶
[ ] متد
[ ] کد وضعیت
[ ] چالش‌ها
[ ] جمع‌بندی

header چیست و با فیلد ناشناخته چه می‌شود؟

هر فیلد header یک جفت «نام: مقدار» است و نام آن به کوچکی و بزرگی حروف حساس نیست؛ فیلد ناشناخته نادیده گرفته می‌شود (RFC 9110 §5.1).

کار فیلد در RFC 9110	نمونه‌ها
مسیریابی	`Host`
دربارهٔ درخواست	`User-Agent` و `Referer`
دربارهٔ پاسخ	`Location` و `Retry-After` و `Allow`
بازنمایی	`Content-Type` و `Content-Encoding` و `Content-Length`

یک نشانی، چند پاسخ؟

- نشانی `/report` یک منبع است که چند **بازنمایی** دارد: HTML برای مرورگر، JSON برای برنامه، و هرکدام با فشرده‌سازی یا بدون آن
- کلاینت ترجیحش را با `Accept-*` می‌گوید؛ سرور انتخاب می‌کند و در پاسخ اعلام می‌کند (مذاکرهٔ محتوا)
- فیلد Vary به cache می‌گوید پاسخ به کدام فیلدهای درخواست وابسته بود (جلسهٔ ۵)

```
GET /report HTTP/1.1
Accept: application/json, text/html;q=0.5
Accept-Encoding: gzip, br, zstd
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Encoding: br
Vary: Accept, Accept-Encoding
```

کلاینت فقط ترجیحش را می‌گوید؛ تصمیم با سرور است و در پاسخ اعلام می‌شود.

زبان سایت را با `Accept-Language` انتخاب کنیم یا با URL جدا؟

سایت دانشگاه نسخهٔ فارسی و انگلیسی دارد. نسخهٔ هر صفحه را چطور انتخاب کنیم؟

الف) با `Accept-Language`

ب) URL جدا: `/fa/` و `/en/`

ج) با IP کاربر

د) با کوکی

زبان سایت را با `Accept-Language` انتخاب کنیم یا با URL جدا؟

سایت دانشگاه نسخهٔ فارسی و انگلیسی دارد. نسخهٔ هر صفحه را چطور انتخاب کنیم؟

الف) با `Accept-Language`

ب) URL جدا: `/fa/` و `/en/`

ج) با IP کاربر

د) با کوکی

پاسخ: ب) با مذاکره، یک URL دو محتوای متفاوت دارد: لینکی که برای دوستتان می‌فرستید ممکن است برای او به زبان دیگری باز شود، و خزندهٔ Google که `Accept-Language` نمی‌فرستد فقط یک نسخه را می‌بیند. راهنمای Google هم URL جدا را توصیه می‌کند.

> **مذاکره برای چیزی است که کاربر به آن فکر نمی‌کند، مثل فشرده‌سازی؛ نه چیزی که می‌خواهد با دیگران به اشتراک بگذارد.**

پیوند با هوش مصنوعی: agent چطور markdown می‌خواهد؟

- agent به منو و اسکریپت HTML نیازی ندارد؛ متن تمیز با توکن کمتر می‌خواهد
- از بهمن ۱۴۰۴، سایتی که در Cloudflare قابلیت Markdown for Agents را روشن کرده باشد، به درخواستی که `Accept: text/markdown` دارد **نسخهٔ markdown** همان صفحه را می‌دهد
- پاسخ `Content-Type: text/markdown` دارد و `Vary: Accept`، تا cache دو نسخه را با هم اشتباه نگیرد

نوع تازه‌ای از کلاینت آمد و سازوکار تازه‌ای لازم نشد؛ مذاکرهٔ محتوای دههٔ ۱۹۹۰ کافی بود.

مذاکره را ببینیم

سؤال یک URL با تغییر `Accept` و `Accept-Encoding` چند پاسخ متفاوت می‌دهد؟

مشاهده فیلدهای `Content-Type` و `Content-Encoding` و `Vary` و طول پاسخ

تصمیم هر جا پاسخ به فیلدی از درخواست وابسته است، `Vary` بفرست

```
U=http://127.0.0.1:8768/report
curl -s -D - -o /dev/null -H 'Accept: text/markdown' $U
curl -s -D - -o /dev/null -H 'Accept: application/json' \
     -H 'Accept-Encoding: gzip' $U
```

پیش از اجرا حدس بزنید: پاسخ JSON با gzip کوچک‌تر می‌شود یا بزرگ‌تر؟

خروجی دمو: یک URL، چهار پاسخ

درخواست	Content-Type	Content-Encoding	طول
`Accept: text/html`	`text/html; charset=utf-8`	—	۳۴ بایت
`Accept: application/json`	`application/json`	—	۳۲ بایت
`Accept: text/markdown`	`text/markdown; charset=utf-8`	—	۲۵ بایت
همان JSON با `Accept-Encoding: gzip`	`application/json`	`gzip`	**۵۳ بایت**

- در هر چهار پاسخ: `Vary: Accept, Accept-Encoding`

بخش ۳ از ۶

متد: تعهدی به کلاینت‌ها و واسطه‌ها

تا اینجا: قالب پیام و فیلدهای header را دیدیم؛ نخستین واژهٔ درخواست، متد است.

سؤال این بخش

متد چه عملی را درخواست می‌کند و چه تعهدی به همراه دارد؟

02-http-transaction.todo

- [x] پیام
- [x] header و مذاکره
- [>] **متد** ▶
- [ ] کد وضعیت
- [ ] چالش‌ها
- [ ] جمع‌بندی

متد چیست و کدام متدها رایج‌اند؟

متد نخستین واژهٔ خط درخواست است و عمل درخواست‌شده روی منبع را مشخص می‌کند (MDN).

متد	عمل درخواست‌شده	نمونهٔ کاربرد
`GET` و `HEAD`	دریافت بازنمایی منبع؛ HEAD بدون بدنه	باز کردن صفحه؛ فرم جستجو با query
`POST`	پردازش داده‌های ارسالی به‌وسیلهٔ منبع	ارسال فرم با داده در بدنه
`PUT`	ایجاد یا جایگزینی کامل منبع در URL معین	بارگذاری فایل در مسیر مشخص
`PATCH`	تغییر بخشی از منبع	ویرایش یک فیلد از پروفایل
`DELETE`	حذف منبع	حذف یک سفارش
`OPTIONS`	پرسش دربارهٔ امکانات منبع	پیش‌درخواست CORS (جلسهٔ ۳)

چه کسانی جز سرور شما به متد نگاه می‌کنند؟

- مرورگر: پیش از ارسال دوبارهٔ فرم POST می‌پرسد، ولی GET را بی‌صدا دوباره می‌فرستد
- پیش‌واکشی، خزنده و اسکنر امنیتی: **لینک‌های GET را خودکار باز می‌کنند**
- حافظهٔ میانی: پاسخ GET و HEAD را معمولاً cache می‌کند، پاسخ POST را نه (جلسهٔ ۵)
- کتابخانهٔ HTTP و proxy: بعضی درخواست‌ها را پس از قطع اتصال خودشان دوباره می‌فرستند

> **متد را فقط برای سرور خودتان انتخاب نمی‌کنید؛ تعهدی است به همهٔ واسطه‌ها و عامل‌های خودکار سر راه.**

safe و idempotent چه فرقی دارند؟

متد	safe	idempotent	کاربرد
`GET` و `HEAD`	✓	✓	خواندن؛ HEAD بدون بدنه
`OPTIONS`	✓	✓	پرسیدن قابلیت‌ها (CORS در جلسهٔ ۳)
`PUT`	✗	✓	جایگزینی کامل در نشانی معلوم
`DELETE`	✗	✓	حذف
`POST` و `PATCH`	✗	✗	پردازش، ساختن، تغییر بخشی از منبع

safe: کلاینت تغییری نخواسته و مسئول اثر جانبی نیست (RFC 9110 §9.2.1) · idempotent: چند بار، همان اثر یک بار (RFC 9110 §9.2.2)

> **safe یعنی «چیزی را عوض نکن»؛ idempotent یعنی «تکرارش چیزی را بیشتر عوض نمی‌کند».**

DELETE دوم 404 می‌گیرد؛ پس idempotent نیست؟

`DELETE /orders/7` بار اول `204` می‌گیرد و بار دوم `404`. پاسخ‌ها با هم فرق دارند.

الف) درست است؛ idempotent نیست

ب) idempotent است

ج) فقط اگر هر دو `204` بودند

د) به cache بستگی دارد

DELETE دوم 404 می‌گیرد؛ پس idempotent نیست؟

`DELETE /orders/7` بار اول `204` می‌گیرد و بار دوم `404`. پاسخ‌ها با هم فرق دارند.

الف) درست است؛ idempotent نیست	**ب)** idempotent است	**ج)** فقط اگر هر دو `204` بودند	**د)** به cache بستگی دارد

پاسخ: ب) idempotent دربارهٔ اثر روی state سرور است، نه یکسان بودن پاسخ. بعد از DELETE اول، سفارش دیگر وجود ندارد و DELETE دوم چیزی را عوض نمی‌کند؛ پس تکرارش بی‌خطر است، هرچند پاسخش فرق دارد.

> **idempotent یعنی تکرار، اثر تازه‌ای ندارد؛ نه اینکه پاسخ‌ها یکی‌اند.**

کدام کار را می‌شود پشت یک لینک ساده (GET) گذاشت؟

۱ جستجوی کتاب در کتابخانهٔ دانشگاه

۲ لغو اشتراک خبرنامه از لینک داخل ایمیل

۳ خروج از حساب کاربری

۴ دیدن فاکتور خرید

الف) فقط ۱ | ب) ۱ و ۴ | ج) ۱ و ۲ و ۴ | د) هر چهار

کدام کار را می‌شود پشت یک لینک ساده (GET) گذاشت؟

۱ جستجوی کتاب در کتابخانهٔ دانشگاه

۲ لغو اشتراک خبرنامه از لینک داخل ایمیل

۳ خروج از حساب کاربری

۴ دیدن فاکتور خرید

الف) فقط ۱	ب) ۱ و ۴	ج) ۱ و ۲ و ۴	د) هر چهار

پاسخ: ب) جستجو و فاکتور فقط می‌خوانند؛ لغو اشتراک و خروج state را عوض می‌کنند. اگر خروج با GET باشد، یک پیش‌واکشی هم کاربر را از حسابش بیرون می‌اندازد.

سناریوی واقعی: چه کسی روی لینک «حذف» کلیک کرد؟

کِی	چه شد	چرا
۱۳۸۴	افزونهٔ Google Web Accelerator صفحه‌های سرویس Backpack را پاک کرد	لینک «delete» با GET کار می‌کرد و افزونه برای سرعت، **لینک‌ها را پیش از کلیک باز می‌کرد**
امروز	اسکنر امنیتی ایمیل، لینک ورود یک‌بارمصرف را پیش از خود کاربر مصرف می‌کند	اسکنر همهٔ لینک‌های نامه را باز می‌کند

راه درست: کاری که چیزی را عوض می‌کند با POST انجام شود. لغو اشتراک یک‌کلیکی در RFC 8058 برای همین با POST است و Gmail آن را از فرستندگان انبوه می‌خواهد.

> **لینکی که چیزی را عوض می‌کند، دیر یا زود یک عامل خودکار آن را باز می‌کند؛ پنجرهٔ تأیید JavaScript هم جلویش را نمی‌گیرد.**

پاسخ نرسید؛ دوباره بفرستیم؟

درخواست رفت و اتصال پیش از رسیدن پاسخ قطع شد. کلاینت نمی‌داند سرور کار را انجام داده یا نه.

- درخواست idempotent را می‌شود **خودکار دوباره فرستاد**؛ اگر بار اول انجام شده باشد، تکرارش چیزی را عوض نمی‌کند (RFC 9110 §9.2.2)
- POST را نه: شاید دو سفارش ثبت شود یا دو بار پول کم شود
- در اینترنت همراه و Wi-Fi ضعیف، این اتفاق هر روز می‌افتد

برای POST هم راهی هست؛ در بخش چالش‌ها.

> **در شبکه، «جواب نیامد» یعنی «نمی‌دانم»، نه «انجام نشد».**

بخش ۴ از ۶

کد وضعیت: پیامی برای نرم‌افزار

تا اینجا: درخواست را ساختیم و دیدیم متد آن چه تعهدی به همراه دارد.

سؤال این بخش

پاسخ را چه کسی می‌خواند، و از یک عدد سه‌رقمی چه می‌فهمد؟

02-http-transaction.todo

- [x] پیام
- [x] header و مذاکره
- [x] متد
- [>] کد وضعیت ▶
- [ ] چالش‌ها
- [ ] جمع‌بندی

از رقم اول کد وضعیت چه می‌شود فهمید؟

کلاس	معنا	کدهای پرکاربرد
1xx	پاسخ موقت؛ پاسخ اصلی در راه است	100 و 101 و 103
2xx	انجام شد	200 و 201 و 204
3xx	برو جای دیگر؛ یا نسخهٔ موجود در cache هنوز معتبر است	301 و 304 و 308
4xx	**اشکال از درخواست**؛ تکرار عینش فایده ندارد، جز 408 و 429	400 و 401 و 403 و 404
5xx	**اشکال از سرور یا واسطه**؛ شاید بعداً درست شود	500 و 502 و 503 و 504

رقم اول می‌گوید تقصیر با کیست و کلاینت چه کند؛ دو رقم بعدی جزئیات است.

کلاینت با کدی که نمی‌شناسد چه می‌کند؟

قاعدهٔ RFC 9110 §15: کلاینت باید **کلاس هر کد را بفهمد** و کد ناشناخته را مثل x00 همان کلاس بداند. کدهای غیراستانداردی که در عمل می‌بینید:

کد	فرستنده	معنا
529	API مدل Claude	سرور بیش از حد شلوغ است
499	nginx، فقط در لاگ	کلاینت پیش از پاسخ اتصال را بست
520 تا 527	Cloudflare	خطا در ارتباط Cloudflare با سرور اصلی

> **کد تازه، کلاینت قدیمی را از کار نمی‌اندازد؛ HTTP با همین قاعده توسعه‌پذیر ماند.**

خطاهای 4xx هرکدام چه می‌گویند؟

کد	معنا	نمونه
`400 Bad Request`	پیام درخواست نادرست است	خط درخواست ناقص، `Host` نیامده
`401 Unauthorized`	احراز هویت نشده‌اید	ورود نکرده‌اید (جلسهٔ ۳)
`403 Forbidden`	شما را می‌شناسد ولی اجازه ندارید	کاربر عادی و صفحهٔ مدیریت
`404 Not Found`	منبعی با این URL نیست	لینک شکسته
`405 Method Not Allowed`	این متد روی این منبع مجاز نیست	`DELETE /`
`429 Too Many Requests`	درخواست‌ها از حد مجاز گذشته است	تلاش پیاپی برای ورود

سناریوی واقعی: مخزن خصوصی؛ 401 یا 403 یا 404؟

کاربری که وارد نشده، اطلاعات یک مخزن **خصوصی** را از API سایت GitHub می‌خواهد. GitHub چه کدی برمی‌گرداند؟

الف) `401`

ب) `403`

ج) `404`

د) `410`

سناریوی واقعی: مخزن خصوصی؛ 401 یا 403 یا 404؟

کاربری که وارد نشده، اطلاعات یک مخزن **خصوصی** را از API سایت GitHub می‌خواهد. GitHub چه کدی برمی‌گرداند؟

الف) `401`

ب) `403`

ج) `404`

د) `410`

پاسخ: ج) `401` یعنی «نمی‌دانم کیستی» و `403` یعنی «می‌دانم کیستی ولی اجازه نداری»؛ هر دو لو می‌دهند که چنین مخزنی هست. GitHub برای همین `404` می‌دهد، و RFC 9110 §15.5.4 هم اجازه می‌دهد سرور برای پنهان کردن وجود یک منبع، به‌جای `403` پاسخ `404` بدهد.

> **کد وضعیت هم چیزی را لو می‌دهد؛ گاهی امن‌ترین پاسخ «چنین چیزی نیست» است.**

پنج کد redirect؛ کدام متد را نگه می‌دارد؟

کد	دائمی؟	متد و بدنه در درخواست بعدی	کاربرد
301	بله	مرورگر POST را GET می‌کند	انتقال دائمی صفحه
302	نه	مرورگر POST را GET می‌کند	انتقال موقت صفحه
303	—	GET	بعد از فرم: برو صفحهٔ نتیجه را بخوان
307	نه	**حفظ می‌شود**	انتقال موقت API
308	بله	**حفظ می‌شود**	انتقال دائمی API

چرا دو کد برای یک کار؟ اسلاید تکمیلی آخر بخش چالش‌ها · redirect در MDN

فیلد `Location` می‌گوید کجا؛ کد می‌گوید تا کِی و با چه متدی.

نتیجهٔ آزمایش: متد و بدنه بعد از redirect

درخواست اول	کد	درخواست دوم
`POST` با `x=1`	`301` یا `302` یا `303`	`GET` بدون بدنه
`POST` با `x=1`	`307` یا `308`	`POST` با `x=1`
`POST` با `--post301`	`301`	`POST` با `x=1`
`POST -X` و `-L`	`301`	`POST` **بدون بدنه**

- سطر آخر دام ابزار است، نه پروتکل: `-X` متد را برای همهٔ درخواست‌ها ثابت می‌کند، ولی curl بدنه را باز هم حذف می‌کند؛ سرور یک POST خالی می‌گیرد
- این‌ها رفتار curl است؛ رفتار مرورگر را استاندارد Fetch تعیین می‌کند و با سه سطر اول یکی است

بخش ۵ از ۶

چالش‌ها: وقتی قاعده‌ها رعایت نشود

تا اینجا: اصل مطلب را دیدیم: پیام و متد و کد وضعیت و header.

سؤال این بخش

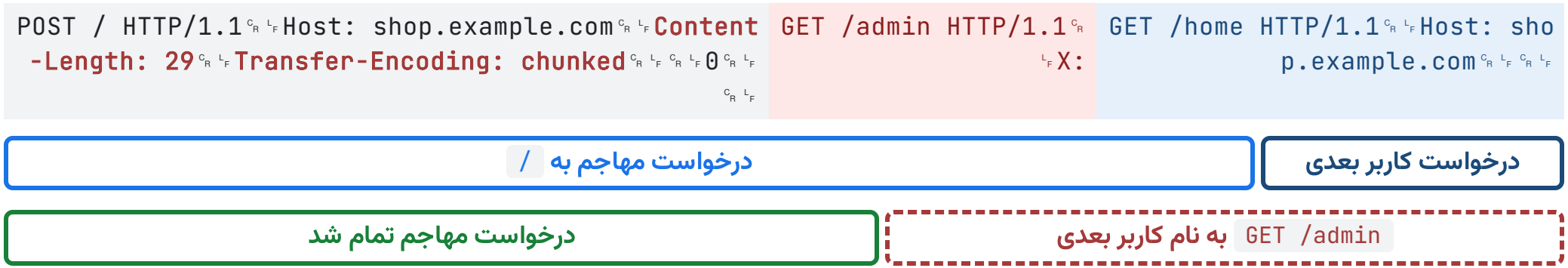
اگر هرکدام از این قاعده‌ها رعایت نشود، چه چیزی خراب می‌شود؟

02-http-transaction.todo

- [x] پیام
- [x] header و مذاکره
- [x] متد
- [x] کد وضعیت
- [>] چالش‌ها ▶
- [ ] جمع‌بندی

از بخش ۱: در HTTP/1.1 مرز پیام را خود پیام اعلام می‌کند.

سناریوی واقعی: proxy و سرور، مرز پیام را دو جور می‌خوانند



- proxy هیچ‌وقت `/admin` را ندید؛ سرور آن را **به نام کاربر بعدی** اجرا می‌کند و پاسخش را به همان کاربر می‌دهد
- اسم این حمله request smuggling است؛ پژوهشی در ۱۴۰۴ با آن ۳۵۰ هزار دلار جایزه گرفت

> **ابهام در قالب انتقال، حفرهٔ امنیتی است؛ راه‌حل: proxy با HTTP/2 به سرور وصل شود.**

از بخش ۳: POST را نمی‌شود خودکار تکرار کرد.

پاسخ پرداخت نرسید؛ چطور بی‌خطر دوباره بفرستیم؟

```
POST /v1/payments HTTP/1.1
Idempotency-Key: "8e03978e-40d5-43e8-bc93-6894a57f9324"
Content-Type: application/json
```

- کلاینت برای هر عملیات یک کلید یکتا می‌سازد و در هر تکرار **همان کلید** را می‌فرستد؛ سرور برای کلید تکراری، نتیجهٔ بار اول را برمی‌گرداند
- Stripe و بسیاری از درگاه‌های پرداخت همین را دارند و پیش‌نویس IETF می‌خواهد استانداردش کند
- همان کلید با بدنهٔ دیگر؟ طبق پیش‌نویس، پاسخ `422`

> **وقتی متد idempotent نیست، idempotency را خودتان با یک کلید بسازید.**

از بخش ۴: کد وضعیت می‌گوید چه شد، نه اینکه چه کسی گفت.

سناریوی واقعی: این 403 را چه کسی فرستاده؟

در سال ۱۳۹۲ درخواست HTTP به سایت فیلترشده در ایران پاسخ `403` می‌گرفت، با یک iframe به `10.10.34.34`. این پاسخ را چه کسی فرستاده بود؟

الف) خود سایت

ب) مرورگر

ج) جعبه‌ای میان راه

د) resolver DNS

از بخش ۴: کد وضعیت می‌گوید چه شد، نه اینکه چه کسی گفت.

سناریوی واقعی: این 403 را چه کسی فرستاده؟

در سال ۱۳۹۲ درخواست HTTP به سایت فیلترشده در ایران پاسخ `403` می‌گرفت، با یک iframe به `10.10.34.34` . این پاسخ را چه کسی فرستاده بود؟

الف) خود سایت	ب) مرورگر	ج) جعبه‌ای میان راه	د) resolver DNS

پاسخ: ج) تجهیزات فیلترینگ درخواست بی‌رمز را از روی `Host` تشخیص می‌دادند و خودشان جواب می‌دادند. در HTTP بی‌رمز کد وضعیت امضا ندارد. TLS جعل پاسخ را ناممکن می‌کند، ولی جلوی مسدود کردن را نمی‌گیرد (جلسهٔ ۴).

> **پاسخ بدون TLS فقط ادعا می‌کند از سرور آمده است.**

از بخش ۲: کلاینت ترجیحش را می‌گوید و سرور انتخاب می‌کند.

سناریوی واقعی: چرا بعد از به‌روزرسانی Chrome صفحه سفید شد؟

- اسفند ۱۴۰۲: Chrome 123 به‌طور پیش‌فرض `zstd` را به `Accept-Encoding` اضافه کرد
- سرور kanidm پاسخ را با zstd فشرده می‌کرد، ولی با پنجره‌ای بزرگ‌تر از ۸ مگابایت که Chrome نمی‌پذیرفت؛ کاربران **صفحهٔ سفید** می‌دیدند (kanidm #2593)
- بعدها RFC 9659 همین سقف ۸ مگابایت را برای zstd در HTTP الزامی کرد؛ حق با Chrome بود
- راه موقت: proxy جلوی سرور، `zstd` را از `Accept-Encoding` حذف کرد

فهرست کلاینت بی‌خبر عوض می‌شود؛ سرور فقط فشرده‌سازی‌ای را انتخاب کند که درست پیاده‌اش کرده است.

از بخش ۳: safe و idempotent دو تعهد جدا هستند.

پیوند با هوش مصنوعی: agent کدام ابزار را تکرار کند؟

در MCP هر ابزار برچسب خطر دارد؛ همان تعهدهای HTTP با نامی تازه.

برچسب ابزار	معادل HTTP	پیش‌فرض
`readOnlyHint`	safe	نه
`idempotentHint`	idempotent	نه
`destructiveHint`	ناامن و مخرب، مثل DELETE	بله

پیش‌فرض‌ها بدبینانه‌اند و برچسب سرور ناشناس را **نباید باور کرد** (MCP blog).

> **agent هم تکرار می‌کند؛ همان دو تعهد HTTP حالا برچسب ابزارهای agent است.**

از بخش‌های ۳ و ۴: idempotent بودن عملیات، و کلاس کد وضعیت.

سناریوی واقعی: agent شما کد ۵۲۹ گرفته است؛ چه کند؟

agent شما برای تولید متن یک `POST` فرستاده و کد غیراستاندارد `529` گرفته است.

الف) تکرار نکند

ب) با فاصلهٔ فزاینده تکرار کند

ج) فوراً ده بار تکرار کند

د) درخواست را اصلاح کند

از بخش‌های ۳ و ۴: idempotent بودن عملیات، و کلاس کد وضعیت.

سناریوی واقعی: agent شما کد ۵۲۹ گرفته است؛ چه کند؟

agent شما برای تولید متن یک `POST` فرستاده و کد غیراستاندارد `529` گرفته است.

الف) تکرار نکند	**ب)** با فاصلهٔ فزاینده تکرار کند	**ج)** فوراً ده بار تکرار کند	**د)** درخواست را اصلاح کند

پاسخ: ب) `529` هم‌ارز `500` است و ممکن است بعداً برطرف شود؛ اگر `Retry-After` آمده باشد، به همان اندازه صبر کند. تولید متن اثر جانبی ندارد و تکرار آن بی‌خطر است؛ ولی تکرار ابزاری با اثر جانبی به کلید idempotency نیاز دارد.

پیش از تکرار: کلاس کد چیست؟ عملیات idempotent است؟ سرور زمانی تعیین کرده است؟

تکمیلی بخش ۴: پنج کد redirect

چرا 307 و 308 لازم شدند؟

- قرار بود `301` و `302` متد را حفظ کنند؛ ولی مرورگرهای دههٔ ۱۹۹۰ بعد از آن‌ها **POST را GET کردند**
- مشخصات رفتار واقعی را پذیرفت: HTTP/1.1 کد `303` را برای «حتماً GET» و `307` را برای «حتماً همان متد» آورد، و RFC 7538 در ۱۳۹۴ کد `308` را
- امروز RFC 9110 §15.4.2 می‌گوید «به دلایل تاریخی» کلاینت می‌تواند بعد از `301` روش POST را GET کند، و استاندارد Fetch می‌گوید مرورگر دقیقاً همین کار را می‌کند

> **همان الگوی جلسهٔ صفر: وقتی رفتار واقعی با مشخصات نخواند، معمولاً مشخصات عوض می‌شود.**

تکمیلی بخش ۴: کلاس 1xx

یک درخواست، چند پاسخ؟

- `103 Early Hints` : تا سرور HTML را آماده کند، به مرورگر می‌گوید CSS و فونت را از همین حالا بگیرد؛ جانشین HTTP/2 Server Push که از Chrome 106 حذف شد (جلسهٔ ۴)
- `101 Switching Protocols` : از این به بعد پروتکل دیگری روی همین اتصال؛ راه ورود WebSocket (جلسهٔ ۶)
- سراغ `100 Continue` در سؤال‌های کنجکاوی می‌رویم

> **یک درخواست می‌تواند چند پاسخ 1xx بگیرد، ولی فقط یک پاسخ نهایی.**

تکمیلی بخش ۲: نام فیلدها

پیشوند X- چرا کنار گذاشته شد؟

- فیلد آزمایشی را با `X-` شروع می‌کردند؛ ولی وقتی استاندارد می‌شد، **نامش دیگر عوض نمی‌شد**
- نمونه: X-Forwarded-For هنوز همه‌جا هست، با اینکه `Forwarded` سال‌هاست جانشین استاندارد آن است
- RFC 6648 در ۱۳۹۱ این رسم را کنار گذاشت؛ نام درست مثل `Idempotency-Key`
- عادت هنوز مانده: Cloudflare فیلد تعداد توکن در Markdown for Agents را `x-markdown-tokens` نامید

> **موقت‌ها ماندگار می‌شوند؛ از روز اول نام درست بگذارید.**

بخش ۶ از ۶

جمع‌بندی

تا اینجا: اصل مطلب و چالش‌های آن را دیدیم.

سؤال این بخش

از هر بخش چه تصمیم مهندسی‌ای باقی می‌ماند؟

02-http-transaction.todo

- [x] پیام
- [x] header و مذاکره
- [x] متد
- [x] کد وضعیت
- [x] چالش‌ها
- [>] **جمع‌بندی** ▶

از امروز کدام تصمیم‌ها را بهتر می‌گیرید؟

مدل ذهنی	تصمیم مهندسی
معنا ثابت است؛ قالب انتقال عوض می‌شود و دام خودش را دارد	اتصال proxy به سرور با HTTP/2؛ تکیه نکردن به reason phrase
متد تعهدی است به همهٔ واسطه‌ها و عامل‌های خودکار	کار تغییردهنده با POST، نه لینک GET؛ کلید idempotency برای تکرار POST
کلاینت کلاس کد را می‌فهمد، نه همهٔ کدها را	`308` برای جابه‌جایی دائمی API؛ `404` برای منبع خصوصی
یک منبع چند بازنمایی دارد	URL جدا برای زبان؛ `Vary` برای هر پاسخ وابسته به درخواست

جلسهٔ بعد: پروتکلی که هر درخواست را جدا می‌بیند، شما را چطور به یاد می‌آورد؟

تمرین ۲: سروری که HTTP را خودش می‌خواند

چالش یک سرور HTTP/1.1 فقط با socket بنویسید که پیام را خودش از بایت‌ها جدا کند: `Host` اجباری، `405` و `308`، و رد درخواستی که مرزش مبهم است. یک شمارندهٔ بازدید هم بدون کوکی بسازید.

سؤال برای درخواستی که هم `Content-Length` دارد و هم `Transfer-Encoding`، RFC 9112 دو راه گذاشته: رد کردن، یا خواندن فقط با chunked. چرا در این تمرین راه سخت‌گیرانه را می‌خواهیم؟

پل شمارندهٔ بدون کوکی جایی اشتباه می‌شمارد؛ جلسهٔ ۳ از همان‌جا شروع می‌شود.

تا جمعه ۸ آبان، ۲۳:۵۹

ie1405-scu.github.io/exercises/02-socket-to-tls/

نقشهٔ فصل ۱: کجای راهیم؟

ch1/syllabus.todo

- [x] جلسهٔ ۱: از URL تا سرور — URL و Origin و DNS، و TLS از دور
- [>] **جلسهٔ ۲: یک تراکنش HTTP** — **متد و کد وضعیت و header** ▶
- [ ] جلسهٔ ۳: state و هویت — cookie و CORS و CSRF
- [ ] جلسهٔ ۴: زیربنای HTTP — TCP و QUIC و TLS
- [ ] جلسهٔ ۵: واسطه‌ها — cache و proxy و CDN
- [ ] جلسهٔ ۶: فراتر از request/response — SSE و WebSocket
- [ ] جلسهٔ ۷: کلاینت‌های خودکار — agent و MCP

هر جلسه یک حلقه از همان زنجیرهٔ ابتدای فصل را بررسی می‌کند.

برای کنجکاوی بیشتر (۱ از ۲)

۱ **چرا نام فیلد `Referer` غلط املایی دارد، و چرا هرگز درست نشد؟**

سرنخ: در RFC 1945 دنبالش بگردید؛ الگوی «هرچه پرکاربردتر، عوض کردنش سخت‌تر» از جلسهٔ صفر.

۲ **چرا HTTP/2 نام فیلدها را حتماً با حروف کوچک می‌خواهد، و سرور با نام بزرگ چه باید بکند؟**

سرنخ: RFC 9113 بخش ۸.۲.

۳ **چرا پاسخ `HEAD` می‌تواند `Content-Length` داشته باشد ولی بدنه نه، و این برای parser کلاینت چه دردسری دارد؟**

سرنخ: قاعدهٔ اول RFC 9112 بخش ۶.۳.

برای کنجکاوی بیشتر (۲ از ۲)

۴ **درخواست `OPTIONS *` دربارهٔ کدام منبع سؤال می‌کند؟**

سرنخ: فرم asterisk در RFC 9112 بخش ۳.۲.۴.

۵ **اگر کلاینت بخواهد بدنهٔ ۲ گیگابایتی بفرستد و سرور آن را نخواهد، سرور چطور پیش از ارسال به او بگوید؟**

سرنخ: `Expect: 100-continue` در RFC 9110 بخش ۱۰.۱.۱.

۶ **کد `418` از کجا آمده، و چرا وقتی خواستند حذفش کنند، ماند؟**

سرنخ: RFC 2324 و جنبش «Save 418»؛ ببینید RFC 9110 با آن چه کرد.

منابع

- منبع درسی: کتاب *Computer Networking* نوشتهٔ Kurose و Ross، بخش‌های ۲.۱.۵ و ۲.۲.۱ و ۲.۲.۳
- MDN: An overview of HTTP · HTTP messages
- RFC 9110: HTTP Semantics
- RFC 9112: HTTP/1.1 · RFC 9113 · RFC 9114
- WHATWG Fetch: HTTP-redirect fetch
- RFC 7538: 308 · RFC 6648 · RFC 8058 · RFC 9659
- PortSwigger: HTTP/1.1 must die · request smuggling
- Signal vs. Noise: Web Accelerator
- Stytch: email scanners · Gmail sender guidelines
- IETF: Idempotency-Key draft · Stripe
- MCP: tool annotations
- Claude API errors
- GitHub REST API: troubleshooting
- Chrome: removing Server Push
- Aryan et al., FOCI 2013
- Google: multilingual sites
- kanidm #2593: zstd
- Cloudflare: Markdown for Agents · blog